LLM, Manual, and Search-Based REST API Test Generation: A Ground-Truth Comparison on Pre-Seeded Faults

Abstract. Large Language Models can generate REST API tests from an OpenAPI specification, and a growing literature reports that they find more errors than search-based tools. Surveying 59 papers (2018–2026), we find no study comparing an LLM against both a manual suite and a mature tool, none computing fault-detection Recall against a known fault population, and none reporting a per-endpoint edge-case metric.
We compare three arms blind: an LLM (Claude Sonnet 4.6), an independently authored manual EP/BVA suite, and EvoMaster 6.0.0. The subjects are three EvoMaster Benchmark APIs covering 35 operations, into which we seed 133 faults by mutation, fixing the denominator of Recall exactly.
The LLM attains 100.0% endpoint coverage and writes 217 edge-case scenarios to the manual suite's 113. Ground-truth Recall nonetheless orders the arms EvoMaster 0.1353 > LLM 0.0602 > manual 0.0150. The ordering follows from oracle kind: a specification-derived status oracle cannot see a fault that changes a computed value while preserving the HTTP status. Scenario counts therefore mislead when reported without a ground-truth Recall beside them.

Keywords: REST API testing · Large language models · Mutation testing · Test oracles · Empirical software engineering

1 Introduction

REST APIs carry most of the traffic between modern software components, and testing them well is expensive and still largely manual. LLMs that read an OpenAPI specification and emit executable tests are therefore of obvious commercial interest, and the research response has been quick. RESTGPT [6] extracts input-constraint rules at 97% precision. AutoRestTest [12] reports 42 server errors against EvoMaster's 20, LlamaRestTest [5] reports 204 faults against 130, and RESTifAI [11] reaches 95.5% operation coverage.

These numbers answer a narrower question than they appear to. Each counts what a suite found on a live system, and none expresses that count as a fraction of what was available to find. Without a known fault population, "42 versus 20" cannot distinguish a suite that found 42 of 50 faults from one that found 42 of 500. For a QA lead deciding whether to trust a generator, the missing denominator is the whole question.

1.1 Gaps

Our systematic review of 59 papers (2018–2026, five independent searches merged into `team-synthesis/evidence-table-merged.md`) isolates three persistent gaps.

GAP-C (Comparison). Only one study compares a generator against a manual suite: EvoMaster [1], which found its own tool below manual (41% vs 82% statement coverage). That study predates LLMs. No paper places an LLM, a manual suite and a mature tool on the same APIs, so the field's central practical claim, that LLM tests rival human ones, has never been tested directly.

GAP-D (Dataset / ground truth). Fault detection is universally reported as raw 500-error counts on live APIs [5,12,2,7,1,8]. With no shared pre-seeded-fault benchmark, ground-truth Recall is never computed.

GAP-M (Metric). Evaluation is coverage-biased. Only KAT [13] splits 2xx from 4xx and only RESTSpecIT [4] reports 5xx. No work reports a per-endpoint edge-case metric or an endpoint-type miss profile.

1.2 This Paper

We close GAP-C and GAP-D together. Seeding faults with standard mutation operators into three EMB APIs fixes the denominator at 133 known faults, and we then run all three arms against every mutant. For GAP-M we add a per-endpoint edge-case count and a code-derived *error-surface baseline*: a static enumeration, per endpoint, of every source location able to emit a non-2xx response. This gives a tool-independent map of what can fail, against which each arm's behaviour is read.

The outcome divides along an unexpected line. On the metrics the literature favours, the LLM wins by a wide margin, with 100.0% endpoint coverage and 217 edge-case scenarios against the manual suite's 113 ($p = 1.43 \times 10^{-7}$). On the ground truth the ranking is EvoMaster 0.1353 > LLM 0.0602 > manual 0.0150. The LLM beats an independently authored human-style suite and is beaten by an established search-based tool, writing roughly 217 scenarios while detecting fewer faults than that tool.

Section 5 traces this dissociation to two mechanisms that compose. Oracle *kind* sets a ceiling, because the spec-derived status-code assertions written by the LLM and by the manual suite cannot see a fault that changes a computed value while leaving the status untouched, which is what most of our mutants do. Within that status-oracle regime, volume decides the rest. Together the two levels explain how a suite can be more thorough by every published metric, better than a human-style baseline, and still less effective than a mature tool.

1.3 Contributions

1. **The first three-way comparison** of LLM, manual and EvoMaster test generation on identical APIs under a blind protocol (GAP-C).

Table 1. Representative studies from the 59-paper merge. Every fault figure in the right-hand column is a count on a live system; none is a Recall against a known fault population.

Study (year)	Generator	Dataset	Metric	Headline result
RESTGPT [6] (2024)	GPT-3.5	9 real APIs	rule P/R/F1	97% rule precision; 72.7% valid inputs
KAT [13] (2024)	GPT-3.5-turbo	12 services	status-code cov.	74.9% (+15.7 pp)
RESTSpecIT [4] (2024)	DeepSeek / GPT	10 PRAB APIs	route disc., 5xx	88.6% routes; 5xx in 4 APIs
AutoRestTest [12] (2025)	GPT-3.5 + MARL	12 services	code cov., 500s	58.3% line; 42 500s vs EvoMaster 20
LlamaRestTest [5] (2025)	Llama3-8B (ft)	12 services	code cov., faults	55.8% method; 204 faults vs 130
LogiAgent [2] (2025)	GPT-4o-mini	12 systems	cov., logical bugs	71.78% line; 49 crashes
RESTifAI [11] (2026)	GPT-4.1-mini	7 services	operation cov.	128/134 ops ≈ 95.5%
EvoMaster [1] (2019)	search-based	3 services	stmt. cov., **vs manual**	38 bugs; 41% generated vs **82% manual**
Morest [8] (2022)	model-based	6 projects	cov., bugs	44 bugs (13 new)
No-Time-to-Rest [7] (2022)	10 tools	20 EMB svcs	line/branch, 500s	EvoMaster-WB 52.76% line

2. **A pre-seeded-fault benchmark** of 133 catalogued mutants over 35 operations, enabling true fault-detection Recall rather than live-API error counts (GAP-D).
3. **A per-endpoint edge-case metric and a code-derived error-surface baseline**, which separate scenarios produced from faults detected (GAP-M).
4. **A two-level mechanism** in which oracle kind sets the ceiling and oracle volume allocates what falls under it, explaining why volume and effectiveness diverge and why scenario-count metrics mislead when reported alone.
5. **A reproducible pipeline** in which every number in this paper is generated from committed raw data by a script rather than transcribed (Section 3.6).

2 Related Work

Our evidence base is a five-member systematic review merged into 59 unique papers (2018–2026) after de-duplication, screened against pre-registered inclusion and exclusion criteria with a PRISMA flow recorded per member. Table 1 lists the ten representative studies that define the state of the art.

Table 2. Gap map: what the 59-paper base does and does not establish.

Gap	Evidence	Status
GAP-C	[1] is the only manual comparison (non-LLM, scored below manual); [5,12,7] automated-only	Confirmed
GAP-D	[5,12,2,7,1,8] report counts, never Recall	Confirmed
GAP-M	[13] (2xx/4xx) and [4] (5xx) partial; none per-endpoint	Confirmed
GAP-T	open-source / non-GPT models sporadic	Deferred (ablation)

2.1 Four Patterns

1. GPT dominates the LLM line. Nearly every LLM study uses an OpenAI model [13,4,6,12,2,11,10], and open-source models appear only sporadically [5]. Our use of Claude Sonnet 4.6 therefore adds a data point outside the family that supplies almost all current evidence, which is a partial contribution to GAP-T.

2. Metrics are coverage plus error counts. Semantic and edge-case quality is rarely isolated. Only KAT [13] splits 2xx from 4xx, and only REST-SpecIT [4] reports 5xx. No study reports edge cases per endpoint, so there is no baseline against which a suite's negative-testing breadth can be judged.

3. Baselines are automated-only. LLM tools are compared against EvoMaster or Morest and reported to beat them on server errors [5,12]. No LLM study includes a manual suite. The single manual comparison in the field remains EvoMaster's own [1], which found the tool well below manual (41% vs 82%). No subsequent LLM paper has revisited that comparison, even as the field's claims moved toward human parity.

4. Fault detection has no denominator. Every fault figure in Table 1 is a count on a live API [5,12,2,7,1,8]. We regard this as the most consequential methodological problem in the area, since error counts and Recall can be ordered differently: a suite that hammers one endpoint hard may log many 500s while missing most of the fault population. Mutation testing is standard elsewhere in software engineering [3,9], and its absence here leaves the comparative claims untestable.

2.2 Positioning

Table 2 maps each gap to the evidence establishing it. Our design targets GAP-C and GAP-D jointly, since adding both a manual arm and a known fault population is what makes the three-way comparison meaningful, and treats GAP-M as the metric facet through which the result is interpreted.

3 Method

The research questions, metrics, thresholds and statistical tests below were fixed in a proposal frozen at instructor approval in Week 5, before any full-run data were seen. RQ2's result contradicts the hypothesis we stated there.

3.1 Research Questions and Hypotheses

RQ1 (absolute). Do LLM-generated tests reach endpoint coverage $\geq 90\%$, and which endpoint types are most often missed? $H_0 : \mu \leq 0.90$; $H_1 : \mu > 0.90$. The 90% threshold is bracketed by the literature, which spans 88.6% route discovery [4] at the low end and 95.5% operation coverage [11] at the high end. *Test:* one-sample Wilcoxon signed-rank against 0.90, one-tailed.

RQ2 (comparative). How many pre-seeded faults do LLM tests detect, versus manual and versus EvoMaster? H_0: $\text{rate}_{\text{LLM}} \leq \max(\text{rate}_{\text{manual}}, \text{rate}_{\text{EvoMaster}})$; H_1: greater than both. *Test:* Friedman across arms, post-hoc pairwise Wilcoxon with Holm correction, and a pooled per-fault McNemar test; effect size Cliff's δ.

RQ3 (comparative). Do LLM tests produce more 4xx/5xx and boundary scenarios per endpoint than manual design? H_0: $\text{median}_{\text{LLM}} \leq \text{median}_{\text{manual}}$. *Test:* paired Wilcoxon signed-rank across endpoints, one-tailed; effect size rank-biserial.

We use $\alpha = 0.05$ per RQ as pre-registered, and additionally report a Bonferroni threshold across the three RQ families ($\alpha_{\text{adj}} \approx 0.0167$) as a stricter second reading.

3.2 Subjects

We use three REST services from the EvoMaster Benchmark (EMB, LGPL-3.0), the shared corpus of the field [7,12]: `rest-ncs` (6 operations), `rest-scs` (11), and `features-service` (18), totalling 35 operations. Each is deployed locally as a standalone JDK 8 Spring Boot fat jar. The harness and EvoMaster run on JDK 17 and communicate with the subjects over HTTP only. Provenance, license and operation counts are recorded in `data/raw/README.md`.

3.3 Fault Seeding and Ground Truth

Faults are seeded by mutation, which supplies the denominator the literature lacks. We apply standard PIT/Offutt operator families [3,9], namely relational replacement, arithmetic replacement and negate/boundary conditionals, to the controller and core-logic classes of each subject, where a fault is observable through the HTTP contract. Each mutation is a single-token change, applied in isolation and recompiled into its own mutant jar. Mutants that fail to compile are discarded, and the pristine source is restored before the next.

This yields 133 compilable mutants: 70 in `rest-ncs`, 59 in `rest-scs` and 4 in `features-service`. Each is catalogued with file, line and operator in `data/faults/<sut>/catalog.json` and exported to `data/full_ground_truth.csv`. The ground truth is derived from code, not from human annotation, so no inter-annotator agreement statistic applies.

The `features-service` imbalance (4 mutants against 70 and 59) is a real property of that service's arithmetic-light controller code rather than a sampling choice. We report it as a generalisation limit instead of repairing it after the fact.

3.4 The Three Arms

All three suites are authored *before* faults are seeded and are blind to the fault catalog, so no arm can target a known mutant.

LLM. Claude Sonnet 4.6 (`claude-sonnet-4-6`), temperature 0, top_p 1, max_tokens 4096, fixed seed. Input is the OpenAPI specification only, so the arm is black-box with no access to source. The prompt is frozen verbatim in `scripts/llm/prompt_template.md`. Per-subject transcripts and the tool-use evidence establishing spec-only input are in `llm/transcripts/`. Output is REST-assured JUnit 5 tests.

Manual. An equivalence-partitioning and boundary-value-analysis suite designed from the same specification and blind to faults, with the design procedure recorded in `manual/methodology.md`. This suite was authored by a separate, isolated model session rather than by a human cohort. We treat that substitution as a construct-validity threat and discuss it in Section 6; throughout this paper the arm is labelled "independently authored" rather than "human".

EvoMaster 6.0.0, black-box mode, seed 42, three repeats:

```
--blackBox true --bbSwaggerUrl <spec>
--outputFormat JAVA_JUNIT_5 --maxTime 120s
```

3.5 Measurement

Coverage is the number of operations exercised by at least one test, divided by the total number of operations, and is computed from the specification together with the executed request log.

Fault Recall is killed/133. An arm kills a mutant if at least one test that *passes* on the original subject *fails* on the mutant, so that the suite observably distinguishes the faulty build from the correct one. Tests already failing on the original are excluded as invalid oracles for that arm, which is why we record `n_oracle` per arm alongside each kill decision.

Edge-case count is the number of distinct negative, boundary and error-code scenarios per endpoint, parsed from `// SCENARIO type=` tags in the suite source.

Error-surface baseline. A static analyser (`scripts/error_surface.py`) enumerates, per endpoint, every source location that can emit a non-2xx response, classified as *declared* (explicit `status(4xx/5xx)`), *framework* (typed `@PathVariable` giving a Spring auto-400), or *potential* (uncaught exception giving a 500). Section 1 describes what this baseline is for.

Error-behaviour key. Separately from the static baseline, we record which distinct (operation, status) behaviours each arm actually elicited at run time (`scripts/error_profile.py`). The denominator of 31 behaviours is the *union* of what the three arms observed, not an independently derived total. An arm that scores 31/31 has therefore observed a superset of the others' behaviours; it has not been shown to cover the full error surface. Section 6 states the consequence for how this metric may be read.

Produced versus detected. Every request each suite issues is replayed through a logging proxy against the live subject and its real HTTP status recorded (`results/raw/traffic.csv`). We then report separately the number of tests and scenarios **produced** and the number of faults and error behaviours actually **detected** (`results/produced-vs-detected.json`). The separation matters because Section 4 shows the two orderings disagree, and conflating them is how a suite that finds fewer faults can be reported as the better one.

3.6 Reproducibility

The pipeline is a chain in which each stage writes a committed artefact:

`mutate.py` → `faults/*/catalog.json` → `run_mutation.py` → `results/raw/*.csv` → `analyze.py` → `results/stats/summary.json` → `gen_paper_macros.py` → this paper

Every computed statistic in this document is a LaTeX macro bound by `gen_paper_macros.py` to a field of `summary.json`, so re-running the experiment moves the paper's text. A small number of fixed integers are written literally, namely the per-subject mutant counts and figures quoted from the Week-7 pilot; `scripts/check_paper.py` fails the build if any *derived* statistic is hardcoded into a results section. We verified that `analyze.py` regenerates `summary.json` byte-identically from the committed raw data, and that the analysis notebook survives Restart & Run All with zero errors. Statistics use `scipy` and `statsmodels`, with the pinned toolchain recorded in `env/tools.md`.

4 Results

4.1 RQ1: Endpoint Coverage

The LLM suite exercises 35 of 35 operations, giving **100.0%** coverage, uniform across all three subjects (`rest-ncs` 6/6, `rest-scs` 11/11, `features-service` 18/18). The one-sample Wilcoxon signed-rank test against the 90% threshold gives $W = 630.0$, $p = 1.65 \times 10^{-9}$, rank-biserial = 1.000. H_0 is **rejected** at $\alpha = 0.05$, and also under the stricter Bonferroni threshold 0.0167.

The pre-registered miss profile by endpoint type is empty, since with every operation covered no endpoint type is missed. This is a real result but a weak one, in that it establishes only that the LLM can reach every documented operation and says nothing about whether it tests any of them well.

4.2 RQ2: Fault Detection against Ground Truth

RQ2 is the pre-registered primary comparison, and it produces a three-way ordering in which the LLM sits in the middle.

Against the 133 seeded faults, overall Recall is **EvoMaster 0.1353**, **LLM 0.0602** and **manual 0.0150**, which in absolute terms is 18, 8 and 2 mutants

Table 3. RQ2: fault-detection Recall per subject and overall, with the independently re-authored manual suite. The overall ordering is EvoMaster > LLM > manual, but it inverts by fault character (Section 5).

Subject	LLM	Manual	EvoMaster
`rest-ncs` (70 mutants)	0.0000	0.0000	**0.1714**
`rest-scs` (59 mutants)	**0.1186**	0.0169	0.0847
`features-service` (4 mutants)	0.2500	0.2500	0.2500
Overall (133 mutants)	0.0602	0.0150	**0.1353**

killed out of 133. EvoMaster detects more than twice the LLM's faults, and the LLM in turn detects four times the independent manual suite's. Our pre-registered H_1, that the LLM would lead both baselines, therefore fails.

The Friedman omnibus test over per-subject Recall gives $\chi^2 = 2.000$, $p = 0.368$ and does not reject. The pre-registered post-hoc pairwise Wilcoxon tests do not reject either: with only three per-subject Recall values per arm, both comparisons give $p_{\text{Holm}} = 1.000$ for LLM-versus-EvoMaster and 1.000 for LLM-versus-manual. We report these because they were pre-registered, but at $n = 3$ subjects this family of tests is severely underpowered, which is why the proposal designated the pooled per-fault McNemar test ($N = 133$) as the primary RQ2 analysis. Both McNemar tests reject.

LLM versus EvoMaster. The discordant pairs are $b = 4$ (killed by the LLM only) against $c = 14$ (EvoMaster only), giving $p = 0.031$, Cliff's $\delta = -0.075$, H_0 **rejected**. EvoMaster significantly out-detects the LLM. Approximate achieved power is 0.557. We return in Section 6 to the fact that this comparison sits close to the α threshold and is sensitive to a single boundary-mutant reproduction.

LLM versus manual. This is the leg the pilot could not interpret, because its manual suite shared an author with the LLM suite. The suite used here was re-authored independently (spec-only, isolated, blind to the LLM output and to the fault catalog), so the comparison is now informative. We find $b = 6$ mutants killed by the LLM only and $c = 0$ by the manual suite only. The manual suite's kills are a strict subset of the LLM's: everything the human-style baseline finds, the LLM also finds, plus six more. With $p = 0.031$, Cliff's $\delta = 0.045$ and H_0 **rejected**, the LLM significantly out-detects an independently authored EP/BVA suite.

Table 3 shows that the overall ordering is not uniform, and that non-uniformity is what Section 5 builds on. On `rest-ncs`, the arithmetic-heavy subject, both status-oracle arms score 0.0000 while EvoMaster kills 0.1714, so the LLM's extra volume gains it nothing there. On `rest-scs`, which carries more validation logic, the LLM leads every arm (0.1186 against manual 0.0169 and EvoMaster 0.0847). On `features-service` all three tie. The ranking follows the fault character of each subject.

Table 4. What each arm produced versus what it detected. The volume ordering (LLM > manual > EvoMaster) is not the effectiveness ordering (EvoMaster > LLM > manual): the arm that produces least detects most.

	LLM	**Manual**	**EvoMaster**
Produced			
Test cases	**295**	147	87
Edge-case scenarios	**217**	113	n/a
Detected (against ground truth)			
Mutants killed / 133	8	2	**18**
HTTP error behaviours / 31	**31**	22	18
5xx responses elicited	**32**	22	18

4.3 RQ3: Edge-Case Scenarios per Endpoint

The LLM produces **217** edge-case scenarios against the manual suite's **113** across 35 operations (median 5.0 versus 3.0 per operation). The paired one-tailed Wilcoxon signed-rank test gives $W = 626.0$, $p = 1.43 \times 10^{-7}$, rank-biserial $= 0.987$. H_0 is **rejected**, and the result survives the Bonferroni threshold 0.0167.

No outlier drives the effect: the LLM produces more scenarios on 33 of 35 operations, manual leads on 2, and 0 tie. This is the largest effect in the study, and as Section 4.4 shows, also the least consequential.

4.4 Produced versus Detected

Table 4 places the volume and effectiveness metrics side by side. The two orderings disagree.

The LLM writes 295 test cases to EvoMaster's 87, which is 3.4× the volume, while EvoMaster detects 2.2× the LLM's faults. The LLM does elicit the widest range of error behaviours, reaching 31 of the 31 behaviours in the pooled key against manual's 22 and EvoMaster's 18, and producing 32 5xx responses including a genuine server crash. Because that key is the union of the three arms (Section 3), this shows the LLM's observed behaviours contain the other arms' and not that it covered the error surface. What the LLM cannot do is detect faults that change a value the specification never constrains.

Had we reported only the volume metrics the surveyed literature uses, the LLM would have ranked first on each of them. It leads EvoMaster on test-case count (295 to 87) and on 5xx responses elicited (32 to 18), and it leads the manual suite on coverage and scenario count. The ground truth agrees about the manual baseline but overturns the verdict against EvoMaster: the tool that produced the fewest tests detected the most faults. Two caveats belong with this comparison. Endpoint coverage and edge-case scenario counts are defined for the LLM and manual arms only, since EvoMaster emits no scenario annotations and we did not instrument it for per-operation coverage, so neither metric ranks all three arms. The comparison against EvoMaster therefore rests on test volume and 5xx tallies, which is precisely the kind of proxy this paper argues against.

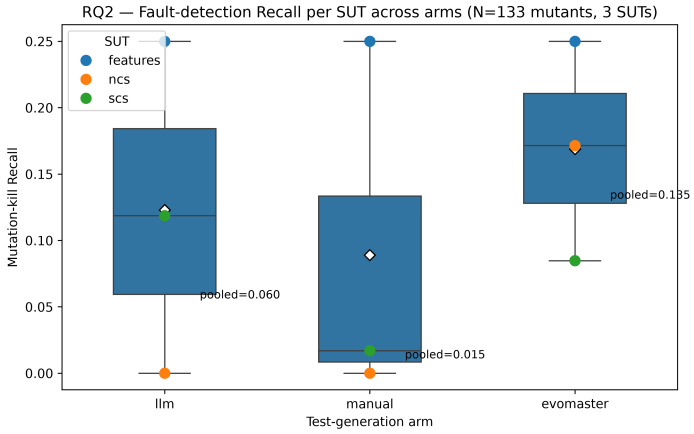

Fig. 1. RQ2: distribution of fault-detection Recall across the three arms over 133 seeded faults. EvoMaster's advantage is concentrated on the arithmetic-heavy `rest-ncs` subject.

5 Discussion

5.1 Why Volume and Effectiveness Came Apart

Section 4 reports a dissociation. The LLM produced 3.4× EvoMaster's test volume, covered every operation, generated 217 edge-case scenarios against the manual suite's 113, and still killed fewer than half as many faults as EvoMaster. The explanation lies in the kind of assertion each arm writes.

The LLM reads an OpenAPI specification, and a specification documents status codes: this path returns 200, that one returns 400 on bad input. Tests synthesised from it therefore assert on the status code and, at best, on the shape

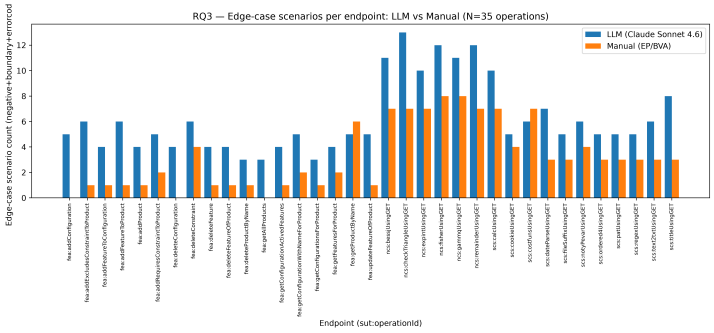

Fig. 2. RQ3: per-endpoint edge-case scenario counts, LLM versus manual across 35 operations. The LLM leads on 33 operations and manual on 2.

of the response body. The resulting oracle says only that the response falls in the plausible class for the request. Our manual EP/BVA suite, drawn from the same specification, writes assertions of the same kind. Both arms are specification-derived status oracles.

Consider what our mutants do to such an oracle. A relational or arithmetic operator replacement inside a numerical routine, such as `<` becoming `<=` or `/` becoming `*`, changes a computed value. The endpoint still returns 200 with a wrong number in the body, so a status oracle passes the mutant and the fault survives.

EvoMaster writes a different kind of oracle. Its black-box mode records the actual response of the original system and emits assertions pinning those recorded values. This is a regression oracle, and it fails whenever any observed value changes, whether or not the status changes with it. Against value-changing mutants it is the right instrument.

The arms were therefore never measuring the same thing. The LLM and manual arms answer whether the API responds in the documented class. EvoMaster answers whether it still computes what it computed before. Our fault population consists almost entirely of value faults, so it bears on the second question. EvoMaster's advantage should be read as evidence that a regression oracle beats a status oracle on value faults, not as evidence that search-based generation is more capable than an LLM.

The per-subject pattern is what this account predicts. If oracle kind drives the result, EvoMaster's advantage should concentrate where faults are most value-like, and it does. EvoMaster leads on `rest-ncs` (0.1714 against 0.0000), whose controllers are numerical routines in which nearly every mutant shifts a computed value while preserving the status. On `rest-scs`, which has more string and validation logic and so more mutants that flip a response class, the LLM leads (0.1186 against 0.0847).

The `rest-ncs` kills make the mechanism concrete. All 12 mutants EvoMaster kills there lie in a single file, `Bessj.java`, an implementation of the Bessel function. 9 of them swap one arithmetic operator for another (`/` for `*`, `-` for `+`) and the remaining 3 alter a numeric comparison. Every one leaves the request valid and the response status at 200, changing only the number returned. That is the exact fault class a status oracle cannot express an assertion about, and it is why both spec-derived arms score 0.0000 on this subject.

Suite size does not explain the gap, and in fact runs against it. On `rest-ncs` the LLM contributes 78 tests that pass on the unmutated subject and therefore qualify as oracles, and the manual suite 50, against EvoMaster's 30. EvoMaster kills 12 mutants with the smallest valid-oracle suite of the three while the two larger suites kill none. Within this subject the relationship between test volume and fault detection is not weak but inverted, which is what a ceiling set by oracle kind looks like from the outside.

The LLM also does better where its oracle is adequate. Where being wrong means returning a different status, it elicits the widest range of behaviours of any arm (31 against manual's 22 and EvoMaster's 18), and it produces a genuine

500 crash. That key is arm-derived, so it cannot tell us how much of the error surface any arm reached (Section 6); it does show that the LLM's greater volume converts into more observed behaviour, but only among faults its oracle could see in the first place.

That gives the two-level picture: oracle kind sets a ceiling and volume distributes what falls under it. The pilot behind this study reported the LLM and manual arms killing identical mutant sets, but its manual suite shared an author with the LLM suite, so that identity could not be interpreted. Re-authoring the manual suite independently separates two effects the tie had conflated. On `rest-ncs`, where every mutant is a value fault, the independent manual suite still scores 0.0000, exactly like the LLM: no status-oracle suite, however authored, can see a wrong number behind a 200. On `rest-scs`, where some mutants flip a response class and are therefore status-visible, the suites separate sharply. The LLM kills 0.1186 to the independent manual's 0.0169, and the manual's kills are a strict subset of the LLM's, so within the status-oracle regime the LLM's 295-test volume reaches more of the catchable faults than the leaner 147-test suite. The pilot's tie was the two levels coinciding through an authorship artefact.

5.2 Implications

For practitioners. An LLM generator gives breadth, not depth. It will reach every documented operation and probe the documented error surface harder than a human or a search tool. It will not notice that a pricing routine now returns the wrong total, because nothing in the specification says what the right total is. The practical rule is to use LLM generation for specification conformance and error-path breadth, and not to let it displace regression oracles, property-based assertions or example-based expected values on computational paths. EvoMaster found 14 faults that the LLM missed entirely.

For the field. Every metric in our study that the surveyed literature actually uses ranked the LLM first; the ground truth ranked it second. We doubt this is peculiar to our subjects, since it is what scoring test suites on volume proxies produces. When AutoRestTest reports 42 server errors against EvoMaster's 20 [12], or LlamaRestTest 204 faults against 130 [5], those numbers are real, but they measure how hard the documented error surface was hit, not what fraction of faults was found. Our results show that the two can order arms oppositely on the same code. The per-endpoint scenario count that GAP-M asks for, and that we introduce, carries the same caveat: RQ3 measures what the LLM writes, and Section 4.4 shows how little that predicted about what it caught.

For benchmark design. The field needs pre-seeded-fault benchmarks because without a denominator the two orderings above are indistinguishable. We would add that a benchmark should stratify its fault population by oracle-observability, separating status-visible from value-only faults, since our results show that single axis reordering the arms. Otherwise the fault mix decides the winner: a benchmark of only value faults will favour regression-oracle tools, and one of only status faults will favour LLMs.

5.3 Why the Low Absolute Recall Is Informative

All three arms score low in absolute terms. The best, EvoMaster, kills 18 of 133 mutants (13.5%). We could have tuned this figure upward by seeding easier faults, and chose not to, because the low ceiling is itself a finding about black-box REST testing. Our mutants live in core logic reachable only through the HTTP contract, and many are equivalent at the boundary: the value they corrupt never surfaces in a response field, or surfaces only for inputs that no arm generated. A white-box tool with code-coverage feedback would score higher. Black-box REST suites, whether written by an LLM, a human or a search algorithm, detect a small minority of seeded logic faults, and the interesting question is which fault classes all three systematically cannot see.

6 Threats to Validity

6.1 Construct Validity

The manual arm is independent-agent, not independent-human. Our proposal specified an independent human baseline. No human cohort was available, so the manual suite was re-authored by a separate, isolated model session with no shared context with the LLM suite, using spec-only input and blind to both the LLM output and the fault catalog. This resolves the pilot's same-author confound, in which one session wrote both suites and they killed identical mutant sets. With the suites separated, the LLM-versus-manual comparison becomes informative and the arms diverge ($b = 6$, $c = 0$; Section 4.2). What remains uncontrolled is the gap between an independent agent and an independent human tester. A human cohort might bring domain intuition that an EP/BVA-from-spec agent lacks, and could plausibly close some of the LLM's lead. We therefore state the RQ2 manual result as "LLM versus an independently authored, specification-derived EP/BVA suite", not "LLM versus human testers". The direction of the conclusion is unaffected, since the LLM's kills are a superset of the manual suite's.

The LLM-versus-EvoMaster significance hinges on one boundary mutant. Our re-execution reproduces the pilot's EvoMaster kills exactly (the same 12 mutants on `rest-ncs`) and the LLM's `rest-scs` kills exactly, but on `rest-ncs` the LLM's Recall falls from the pilot's 1/70 to 0.0000. One LLM test exercising `bessj` at the boundary $n = 2$ passed on the pilot's build of the original SUT but fails on our clean rebuild, so it is excluded as an invalid oracle and no longer kills that mutant. Bessel-function evaluation at the boundary is numerically sensitive, and we regard this as a genuine reproduction limit and not a pipeline error, since the exact EvoMaster match rules out a build fault. Losing that one kill shifts the LLM-versus-EvoMaster discordant count from five to 4, which is what carries the McNemar p across the $\alpha = 0.05$ threshold, from just above it in the pilot to 0.031 here. The direction (EvoMaster > LLM) is robust either way. The $\alpha = 0.05$ verdict is not, and should be read as "EvoMaster leads, at the edge of significance" rather than as a firm rejection.

Oracle strength differs across arms by construction. EvoMaster's recorded-value regression oracles are strictly stronger on value faults than the spec-derived status oracles of the LLM and manual arms. We did not fail to control this; it is the independent variable that Section 5 identifies as driving the result. It does mean, however, that RQ2 does not compare generation techniques with oracle strength held constant. A fairer isolation of generation quality would fix the oracle kind across arms and vary only the generator. We report kill-by-subject so that the interaction stays visible instead of being buried in a pooled average.

The error-behaviour denominator is arm-derived, so its ceiling is not informative. The 31 HTTP error behaviours of Section 4.4 are the union of what the three arms elicited, so the highest-scoring arm reaches 31/31 by construction. The LLM's score on this metric establishes that its observed behaviour set contains the manual suite's and EvoMaster's, and nothing stronger. In particular it is not evidence that the LLM covered the error surface, and it is not comparable to the code-derived static baseline, which counts source locations rather than observed behaviours. A study wanting a true error-surface Recall would have to score every arm against the static enumeration; we did not, and we flag it as the clearest measurement gap in this paper.

Mutation kills are a proxy for real faults. A seeded single-token mutation is not a field defect, and coverage is likewise a proxy for test quality. We report Recall, coverage and edge-case counts jointly, and Section 4.4 exists because any one of them alone gives the wrong ranking.

6.2 Internal Validity

Equivalent mutants are not excluded. We do not attempt to identify semantically equivalent mutants, which is undecidable in general. Some fraction of the 133 mutants may be unkillable by any black-box suite, which deflates all three arms' Recall uniformly. This weakens the absolute figures but not the comparison, since every arm faces the identical population.

LLM non-determinism. Generation used temperature 0 and a fixed seed, and raw outputs are frozen in the repository, but LLM sampling is not bit-reproducible across provider-side model updates. The generated suites are committed, so the analysis is reproducible even where regeneration is not.

Mutation confined to controller and core-logic classes. We seed where faults are observable through the HTTP contract. Faults in configuration, persistence or serialisation layers fall outside the population and outside our conclusions.

6.3 External Validity

Three JVM APIs, 35 operations. Our subjects are small EMB services, so the findings are EMB-only and JVM-only and should not be read as covering large production APIs, authenticated flows or non-JVM stacks. We excluded the `user-management` service (MySQL-backed) on setup cost, leaving authentication

endpoints under-represented, so RQ1's miss profile cannot speak to the endpoint type most likely to be missed.

The `features-service` imbalance. That subject contributes 4 mutants against 70 and 59 for the others, because its controllers contain little arithmetic to mutate. All three arms tie there (0.2500 each), so it contributes almost nothing to the pooled comparison. We keep it in, because dropping a subject after seeing its results would amount to selecting on the outcome.

One LLM, one configuration. We used Claude Sonnet 4.6 at temperature 0 with a single frozen prompt, so our results do not generalise to the GPT family that dominates the literature (Section 2), to open-source models, or to agentic multi-turn strategies. Since the mechanism we identify is oracle kind, we expect the qualitative finding to hold for any spec-only generator. We have not tested that.

6.4 Conclusion Validity

The Friedman test is underpowered by design. With $n = 3$ subjects it cannot detect realistic effects ($\chi^2 = 2.000$, $p = 0.368$), which is why the proposal pre-designated the pooled per-fault McNemar test ($N = 133$) as the primary RQ2 analysis. We did not promote it after the omnibus test failed.

RQ2's pairwise tests are significant but fragile. Both McNemar comparisons reject at $\alpha = 0.05$ ($p = 0.031$ for LLM-versus-EvoMaster and $p = 0.031$ for LLM-versus-manual), but neither clears the Bonferroni threshold 0.0167, and approximate achieved power is below conventional adequacy for both: 0.557 for the EvoMaster comparison and 0.530 for the manual one. We therefore rest the RQ2 conclusions on direction and effect size, namely that EvoMaster kills more than twice as many mutants as the LLM and dominates outright on the value-fault subject, and that the LLM's kills strictly contain the manual suite's. The EvoMaster comparison is not a set containment: $b = 4$ mutants are killed by the LLM alone. We do not rest the conclusions on the p-values alone, and we do not upgrade "significant at 0.05" to "firmly established". The power figure is itself an approximation, since statsmodels has no exact McNemar power routine and we substitute a two-independent-proportion calculation on the same proportions, disclosed in `summary.json` alongside the value.

Multiplicity. Three RQ families are tested. We apply Holm correction within RQ2's post-hoc comparisons and report a Bonferroni threshold across families ($\alpha_{\text{adj}} \approx 0.0167$). RQ1 and RQ3 survive both comfortably ($p = 1.65 \times 10^{-9}$ and $p = 1.43 \times 10^{-7}$). The RQ2 pairwise tests survive the raw threshold but not Bonferroni, which is why we read them as directional evidence and not as decisive rejections.

7 Conclusion

We ran a comparison the REST-API-testing literature has not: an LLM generator, a manual EP/BVA suite and EvoMaster, on the same three APIs, against

a known population of 133 seeded faults rather than a tally of errors observed on a live system.

Can an LLM out-test a human-style baseline and a search-based tool? It out-tests the human-style baseline and is out-tested by the tool. On the volume metrics the field currently reports, the LLM leads: 100.0% endpoint coverage ($p = 1.65 \times 10^{-9}$), 217 edge-case scenarios against the manual suite's 113 ($p = 1.43 \times 10^{-7}$, rank-biserial 0.987), and 295 test cases to EvoMaster's 87. On the ground truth the ordering is EvoMaster 0.1353 > LLM 0.0602 > manual 0.0150. The LLM significantly out-detects an independently authored EP/BVA suite ($p = 0.031$, with its kills a strict superset), yet is itself out-detected by EvoMaster ($p = 0.031$). Both results hold at the raw threshold and neither holds under Bonferroni, so we read them as directional rather than decisive.

The explanation is not about model capability. A generator that reads a specification writes assertions about status codes, and a fault that changes a computed value while leaving the status intact is invisible to every such assertion, no matter how many are written. EvoMaster's recorded-value regression oracles see exactly those faults. The arms' ordering follows fault character: on the arithmetic-heavy `rest-ncs` both status-oracle arms score 0.0000 while EvoMaster kills 0.1714, and on the validation-heavy `rest-scs` the LLM leads every arm. Oracle kind fixes the ceiling; within a kind, volume decides how much of the reachable fault set is caught.

For practitioners, LLM generation complements value-based oracles and does not replace them: it beats a human-style suite yet still missed 14 faults that EvoMaster's regression oracles caught. For the field, the metric that ranks the LLM first over EvoMaster (volume) and the metric that ranks it below (ground-truth Recall) disagree on the same code, so the comparative claims currently in the literature are not measuring what their framing implies. For benchmark design, fault populations should be stratified by oracle-observability, because that single axis reorders the arms.

Our principal remaining limitation is that the independent manual baseline is an isolated agent session rather than a human cohort. A human replication against the same 133 mutants is the highest-value next step, and our pipeline runs it without modification. Beyond that, the natural follow-up studies are an oracle-controlled comparison that fixes assertion kind and varies only the generator, a fault population stratified by oracle-observability, and the GAP-T ablation across model families. The pipeline, prompts, fault catalogs and raw data are committed, and every number above regenerates from them by script.

Use of AI Tools

This study concerns LLM-based test generation, so we separate three distinct roles that language models play in it.

As an object of study. Claude Sonnet 4.6 is the subject of the LLM arm. Its configuration, prompt and outputs are specified in Section 3 and committed to the replication package.

As an instrument in the experiment. The manual EP/BVA arm was authored by a separate, isolated model session rather than by a human cohort, because no human cohort was available. We treat this as a construct-validity threat, label the arm "independently authored" rather than "human" throughout, and discuss its consequences in Section 6.

In manuscript preparation. The authors used an LLM assistant to draft and edit prose, and to convert the manuscript to this template. All research design, experimental execution, data analysis and interpretation were carried out by the authors. Every computed statistic reported here is generated from committed raw data by `scripts/gen_paper_macros.py` and was not written by a model or transcribed by hand (Section 3.6). The authors read, verified and take full responsibility for the entire content of this paper. No LLM is listed as an author, in line with Springer's authorship policy.

Data Availability

The replication package contains the mutation catalogs, the generated test suites for all three arms, the raw kill matrices, the analysis scripts and the notebook that regenerates every statistic in this paper. Subjects are drawn from the EvoMaster Benchmark (EMB, LGPL-3.0); the upstream commit used is recorded in the repository.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Arcuri, A.: RESTful API automated test case generation with EvoMaster. ACM Transactions on Software Engineering and Methodology (TOSEM) **28**(1) (2019). https://doi.org/10.1145/3293455
2. Chen, K., Zhang, Y., Wang, J., et al.: LogiAgent: Automated logical testing for REST systems with LLM-based multi-agents. arXiv preprint (2025). https://doi.org/10.48550/arXiv.2503.15079
3. Coles, H., Laurent, T., Henard, C., Papadakis, M., Ventresque, A.: PIT: A practical mutation testing tool for Java (demo). In: ACM SIGSOFT Int. Symp. on Software Testing and Analysis (ISSTA) (2016). https://doi.org/10.1145/2931037.2948707
4. Decrop, A., Perrouin, G., Papadakis, M., Devroey, X., Schobbens, P.Y.: You can REST now: Automated specification inference and black-box testing of RESTful APIs with large language models. arXiv preprint (2024). https://doi.org/10.48550/arXiv.2402.05102
5. Kim, M., Sinha, S., Orso, A.: LlamaRestTest: Effective REST API testing with small language models. In: ACM Int. Conf. on the Foundations of Software Engineering (FSE) (2025). https://doi.org/10.1145/3715737
6. Kim, M., Stennett, T., Shah, D., Sinha, S., Orso, A.: Leveraging large language models to improve REST API testing. In: IEEE/ACM Int. Conf. on Software Engineering: New Ideas and Emerging Results (ICSE-NIER) (2024). https://doi.org/10.1145/3639476.3639769

7. Kim, M., Xin, Q., Sinha, S., Orso, A.: Automated test generation for REST APIs: No time to rest yet. In: ACM SIGSOFT Int. Symp. on Software Testing and Analysis (ISSTA) (2022). https://doi.org/10.1145/3533767.3534401
8. Liu, Y., Li, Y., Deng, G., et al.: Morest: Model-based RESTful API testing with execution feedback. In: IEEE/ACM Int. Conf. on Software Engineering (ICSE) (2022). https://doi.org/10.1145/3510003.3510133
9. Offutt, A.J., Untch, R.H.: Mutation 2000: Uniting the orthogonal. In: Mutation Testing for the New Century, pp. 34–44. Springer (2001). https://doi.org/10.1007/978-1-4757-5939-6_7
10. Pereira, A., Lima, B., Faria, J.P.: APITestGenie: Automated API test generation through generative AI. In: IEEE/ACM Int. Conf. on Automation of Software Test (AST) (2026). https://doi.org/10.1145/3793654.3793743
11. Schäfer, M., Nguyen, T., et al.: RESTifAI: An LLM workflow for reusable REST API testing. In: IEEE/ACM Int. Conf. on Software Engineering (ICSE), Demonstrations (2026). https://doi.org/10.48550/arXiv.2512.08706
12. Stennett, T., Kim, M., Sinha, S., Orso, A.: AutoRestTest: A tool for automated REST API testing using LLMs and multi-agent reinforcement learning. In: IEEE/ACM Int. Conf. on Software Engineering (ICSE) (2025). https://doi.org/10.1109/ICSE55347.2025.00179
13. Tran, T., Le, D., Nguyen, H., et al.: KAT: Dependency-aware automated API testing with large language models. In: IEEE Conf. on Software Testing, Verification and Validation (ICST) (2024). https://doi.org/10.1109/ICST60714.2024.00017